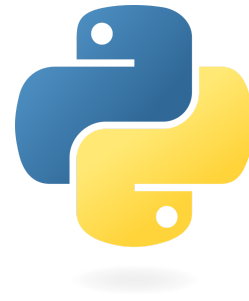


Taller Introducción a Python

DUOC UC - Primer paso



Presentación del Taller

El taller de Introducción a Python está diseñado para estudiantes que desean aprender los fundamentos de la programación con uno de los lenguajes más versátiles y populares en la actualidad. En este taller, exploramos conceptos básicos que son esenciales para iniciar una carrera en ingeniería en informática en **Duoc UC**, donde la práctica y la innovación tecnológica son pilares fundamentales.

Tipos de Datos en Python

En Python, un tipo de dato define la naturaleza de un valor y determina cómo se puede manipular ese valor. Los tipos de datos básicos de Python son altamente versátiles y se ajustan automáticamente según el valor asignado, eliminando la necesidad de especificarlos explícitamente, como ocurre en otros lenguajes de programación.

1. Cadenas de Texto (String, `str`)

Las cadenas de texto son secuencias de caracteres que se utilizan para representar información textual. Pueden contener letras, números y símbolos, y se delimitan con comillas simples (`' '`) o dobles (`" "`). Este tipo de dato es fundamental en la programación, ya que permite manejar nombres, direcciones, mensajes, y cualquier otra información basada en texto.

Características clave:

- Las cadenas son inmutables, es decir, no pueden cambiar después de ser creadas.
- Pueden ser manipuladas con operaciones como concatenación, repetición y acceso por índice.
- Python incluye métodos integrados para trabajar con cadenas, como `upper()`, `lower()`, `replace()`, y más.

Ejemplo práctico:

```
# Crear una cadena
mensaje = "Hola, Python"
print(mensaje)

# Concatenar cadenas
nombre = "Marcelo"
saludo = "Hola, " + nombre + "!"
print(saludo)

# Acceso a caracteres individuales
print(mensaje[0]) # Imprime 'H'

# Métodos útiles
print(mensaje.upper()) # Convierte la cadena a mayúsculas
print(mensaje.replace("Python", "Mundo")) # Reemplaza una parte de la cadena
```

2. Números Enteros (`int`)

Este tipo de dato representa números enteros, positivos o negativos, sin decimales. Son ideales para contar elementos, realizar operaciones matemáticas simples o representar valores discretos.

Características clave:

- Pueden ser utilizados en operaciones aritméticas como suma, resta, multiplicación y división.
- No tienen límite de tamaño (más allá de la memoria disponible en el sistema).
- Se declaran automáticamente al asignar un valor sin punto decimal.

Ejemplo práctico:

```
# Declarar números enteros
edad = 25
dias = 365

# Operaciones matemáticas
suma = edad + 5
producto = dias * 2
print(suma, producto)
```

3. Números Flotantes (`float`)

Representan números con decimales, como `3.14`, `-0.001`, o `2.0`. Este tipo es crucial para cálculos más precisos que involucran fracciones o valores continuos.

Características clave:

- Pueden representarse en notación científica, como `1.5e3` para `1500.0`.
- Se utilizan para cálculos matemáticos que requieren precisión decimal.

- Python permite convertir entre enteros y flotantes fácilmente usando las funciones `int()` y `float()`.

Ejemplo práctico:

```
# Declarar números flotantes
altura = 1.75
peso = 70.5

# Operaciones con decimales
imc = peso / (altura ** 2)
print("Tu IMC es:", round(imc, 2)) # Redondea a 2 decimales
```

4. Booleanos (`bool`)

Los booleanos son un tipo de dato que solo puede tener dos valores: `True` (verdadero) o `False` (falso). Son esenciales para estructuras de control, condicionales y bucles.

Características clave:

- Los valores booleanos se utilizan para representar el estado de una condición lógica.
- Python interpreta automáticamente ciertos valores como `True` o `False` en contextos booleanos, por ejemplo:
 - Números diferentes de 0 → `True`
 - Cadenas no vacías → `True`
 - Listas vacías o el valor `None` → `False`

Ejemplo práctico:

```
#Booleanos
es_estudiante = True
```

```
tiene_permiso = False

# Usar booleanos en condicionales
if es_estudiante and tiene_permiso:
    print("Puedes acceder al laboratorio CITT de DUOC UC.")
else:
    print("No tienes permiso para acceder.")
```

5. Listas y Otros Tipos Compuestos

Aunque no forman parte de los tipos de datos básicos, las listas, tuplas, y diccionarios son estructuras de datos fundamentales en Python. Estas permiten almacenar y manipular múltiples valores en un solo lugar.

Resumen de la flexibilidad de Python:

En Python, no es necesario declarar explícitamente el tipo de dato; el lenguaje lo deduce automáticamente al asignar un valor. Esto hace que trabajar con diferentes tipos de datos sea rápido y eficiente, incluso para principiantes.

Explorar los tipos de datos básicos es el primer paso para comprender cómo interactuar con información en Python. Una base sólida en estos conceptos es esencial para tareas más avanzadas como análisis de datos, desarrollo web, inteligencia artificial y más.

1. Operaciones con Variables

Vimos cómo crear y manipular variables, las cuales son la base de cualquier programa. Aprendimos sobre la asignación de valores a variables, la concatenación de cadenas, y la conversión de tipos de datos (casting)

2. Entrada y Salida de Datos

Usamos las funciones `input()` y `print()` para interactuar con los usuarios, permitiendo recibir datos y mostrar resultados

3. Condicionales

Entendimos cómo tomar decisiones dentro de un programa utilizando estructuras condicionales como `if`, `elif` y `else`. Estas estructuras nos permiten ejecutar diferentes bloques de código según las condiciones que se cumplan o no.

4. Comparaciones y Operadores Lógicos

Los operadores de comparación (como `==`, `!=`, `>`, `<`) y los operadores lógicos (`and`, `or`, `not`) son esenciales para evaluar expresiones y tomar decisiones basadas en múltiples condiciones

5. Bucles

Exploramos los bucles `while` y `for`, que nos permiten ejecutar un bloque de código de manera repetitiva mientras se cumpla una condición o sobre una secuencia de elementos

2 - Operaciones con Variables en Python

Las variables son nombres simbólicos que se utilizan para almacenar datos en la memoria de un programa. Son la base para realizar cualquier operación en Python, ya que permiten guardar valores que pueden cambiar durante la ejecución del código. A continuación, exploramos las operaciones más comunes y cómo se utilizan.

1. Asignación de Variables

La asignación consiste en darle un valor a una variable mediante el operador de asignación `=`. Python permite asignar valores de forma simple o múltiple.

Características clave:

- El tipo de dato de la variable se determina automáticamente según el valor asignado.
- Puedes reasignar valores a las variables tantas veces como sea necesario.

Ejemplo práctico:

```
# Asignación simple
x = 10
y = 20

# Asignación múltiple
a, b, c = 1, 2, 3
print(a, b, c) # Salida: 1 2 3

# Intercambio de valores
a, b = b, a
print(a, b) # Salida: 2 1
```

2. Operaciones Matemáticas

Python soporta una amplia variedad de operaciones matemáticas sobre variables numéricas. Estas incluyen:

- **Suma (+)**: Añadir dos números.
- **Resta (-)**: Restar un número de otro.
- **Multiplicación (*)**: Multiplicar dos números.
- **División (/)**: Dividir un número por otro, devolviendo un flotante.
- **División entera (//)**: Divide y redondea hacia abajo al entero más cercano.
- **Módulo (%)**: Devuelve el resto de una división.
- **Exponenciación (**)**: Eleva un número a la potencia de otro.

Ejemplo práctico:

```
x = 15
y = 4
```

```
print(x + y) # Suma: 19
print(x - y) # Resta: 11
print(x * y) # Multiplicación: 60
print(x / y) # División: 3.75
print(x // y) # División entera: 3
print(x % y) # Módulo: 3
print(x ** y) # Exponenciación: 50625
```

3. Concatenación de Cadenas

Las cadenas de texto se pueden unir (concatenar) usando el operador `+`. Esto permite combinar varios textos en uno solo.

Ejemplo práctico:

```
nombre = "Marcelo"
apellido = "Crisóstomo"
nombre_completo = nombre + " " + apellido
print(nombre_completo) # Salida: Marcelo Crisóstomo
```

También puedes usar el operador `*` para repetir una cadena varias veces:

```
mensaje = "Python "
print(mensaje * 3) # Salida: Python Python Python
```

4. Conversión de Tipos (Casting)

Python permite convertir entre tipos de datos cuando sea necesario. Esto es útil para realizar operaciones entre tipos incompatibles o para garantizar que los datos sean del formato esperado.

Funciones comunes de conversión:

- `int()`: Convierte a entero.
- `float()`: Convierte a número decimal.
- `str()`: Convierte a cadena de texto.

Ejemplo práctico:

```
# Conversión de cadena a número
edad = "25" # Edad como cadena
edad = int(edad) # Convertimos a entero
print(edad + 1) # Salida: 26

# Conversión de número a cadena
num = 100
num_str = str(num)
print("El número es " + num_str) # Salida: El número es 100
```

5. Operaciones de Asignación Compuesta

Python permite simplificar operaciones comunes usando operadores de asignación compuesta, como `+=`, `-=`, `*=`, `/=`, entre otros.

Ejemplo práctico:

```
x = 10
x += 5 # Equivale a x = x + 5
print(x) # Salida: 15

x *= 2 # Equivale a x = x * 2
print(x) # Salida: 30
```

6. F-Strings para Formateo de Cadenas

Introducido en Python 3.6, las f-strings son una forma moderna y eficiente de insertar variables dentro de cadenas de texto.

Ejemplo práctico:

```
nombre = "Ana"
edad = 30
print(f"Hola, {nombre}. Tienes {edad} años.") # Salida: Hola, Ana. Tienes 30 años.
```

Importancia de las Operaciones con Variables

Dominar las operaciones con variables es esencial para construir programas dinámicos y funcionales. Desde la gestión de datos del usuario hasta cálculos complejos, estas operaciones forman la base para cualquier aplicación en Python. Con una comprensión sólida de este tema, estarás preparado para enfrentar desafíos más avanzados en el desarrollo de software.

3 - Entrada y Salida de Datos en Python

La entrada y salida de datos son elementos fundamentales en la programación, ya que permiten interactuar con los usuarios o procesar información de diversas fuentes. Python facilita estas interacciones mediante funciones simples y poderosas como `input()` y `print()`.

1. Entrada de Datos con `input()`

La función `input()` permite capturar información del usuario desde el teclado. Siempre devuelve los datos como una cadena de texto (`str`), independientemente del tipo de dato que se introduzca. Si necesitas trabajar con números u otros tipos de datos, debes realizar una conversión explícita (casting).

Características clave:

- Es interactiva: el programa se detiene hasta que el usuario introduce un valor y presiona Enter.
- Siempre devuelve una cadena (`str`).
- Puede incluir un mensaje para guiar al usuario.

Ejemplo práctico:

```
# Capturar el nombre del usuario
nombre = input("¿Cómo te llamas? ")
print(f"Hola, {nombre}!")

# Capturar un número e integrarlo en cálculos
edad = input("¿Cuántos años tienes? ")
edad = int(edad) # Convertimos la entrada a entero
print(f"El próximo año tendrás {edad + 1} años.")
```

2. Salida de Datos con `print()`

La función `print()` muestra información en la consola para que el usuario la vea. Es extremadamente flexible y puede imprimir cadenas, números, variables y resultados de operaciones matemáticas. Además, permite personalizar la salida mediante separadores (`sep`) y finales (`end`).

Características clave:

- Puede mostrar múltiples elementos separados por comas.
- Permite personalizar cómo termina la línea usando `end` .
- Puede incluir expresiones y valores calculados directamente.

Ejemplo práctico:

```
# Imprimir texto simple
print("Bienvenido al taller de Python.")
```

```
# Imprimir múltiples valores
nombre = "Marcelo"
edad = 30
print("Nombre:", nombre, "Edad:", edad)

# Personalizar la salida
print("Primera línea", end=" | ")
print("Segunda línea") # Salida: Primera línea | Segunda línea
```

3. Formateo de Salidas

Python ofrece varias formas de formatear salidas para que sean más legibles o específicas. Los métodos más utilizados son:

1. Concatenación de cadenas:

Combinar texto usando el operador

`+`.

```
nombre = "Marcelo"
print("Hola, " + nombre + "!")
```

2. F-strings (formato moderno):

Introducido en Python 3.6, permite insertar variables directamente dentro de cadenas utilizando llaves

`{}`.

```
nombre = "Marcelo"
edad = 30
print(f"Hola, {nombre}. Tienes {edad} años.")
```

3. Método `format()`:

Usado en versiones anteriores a Python 3.6.

```
nombre = "Marcelo"
print("Hola, {}".format(nombre))
```

4. Conversión de Datos (Casting) en Entrada

Dado que la función `input()` devuelve cadenas, es necesario convertirlas a otros tipos de datos para operaciones específicas. Las funciones más comunes son:

- `int()`: Convierte a entero.
- `float()`: Convierte a número decimal.
- `str()`: Convierte a cadena.

Ejemplo práctico:

```
# Capturar un número como entero
edad = int(input("¿Cuántos años tienes? "))
print(f"Tu edad el próximo año será {edad + 1}.")

# Capturar un número como flotante
altura = float(input("¿Cuál es tu altura en metros? "))
print(f"Tu altura es {altura} metros.")
```

5. Personalización de `print()` con Parámetros

La función `print()` permite personalizar la salida utilizando los parámetros:

- `sep`: Define el separador entre los elementos.
- `end`: Define cómo termina la línea (por defecto, con un salto de línea).

Ejemplo práctico:

```
# Personalizar separador
print("Python", "es", "genial", sep="-") # Salida: Python-es
-genial
```

```
# Personalizar final de línea
print("Primera línea", end="... ")
print("Segunda línea") # Salida: Primera línea... Segunda línea
```

6. Ejemplo Completo de Entrada y Salida

A continuación, un programa que combina entrada y salida para interactuar con el usuario:

```
# Programa interactivo
print("Bienvenido al sistema interactivo.")
nombre = input("¿Cómo te llamas? ")
edad = int(input("¿Cuántos años tienes? "))
altura = float(input("¿Cuál es tu altura en metros? "))

print("\nResumen de tus datos:")
print(f"Nombre: {nombre}")
print(f"Edad: {edad} años")
print(f"Altura: {altura} metros")
print(f"El próximo año tendrás {edad + 1} años.")
```

Importancia de la Entrada y Salida

La entrada y salida son esenciales para cualquier programa que interactúe con usuarios o procese datos. Estos conceptos permiten crear aplicaciones interactivas, analizar datos, o simplemente mostrar resultados de cálculos.

Dominar estas funciones es un paso clave para el desarrollo de software dinámico y útil.

4- Condicionales

Condicionales en Python

Las condicionales son estructuras de control que permiten ejecutar diferentes bloques de código según si una condición es verdadera o falsa. Son fundamentales para la toma de decisiones en programación, ya que permiten que los programas reaccionen de manera dinámica a diferentes situaciones.

1. ¿Qué son las Condicionales?

Las condicionales evalúan expresiones lógicas y ejecutan instrucciones según el resultado de esa evaluación. En Python, las palabras clave principales para crear estructuras condicionales son:

- `if`: Evalúa una condición. Si es verdadera, ejecuta el bloque de código correspondiente.
- `elif` (opcional): Permite evaluar condiciones adicionales si las anteriores son falsas.
- `else`: Se ejecuta si todas las condiciones anteriores son falsas.

Estructura básica:

```
if condición:
    # Código que se ejecuta si la condición es verdadera
elif otra_condición:
    # Código que se ejecuta si otra_condición es verdadera
else:
    # Código que se ejecuta si ninguna condición anterior es verdadera
```

2. Operadores de Comparación

Para evaluar condiciones, Python utiliza operadores de comparación. Estos incluyen:

- `==`: Es igual a.
- `!=`: No es igual a.

- `<`: Menor que.
- `>`: Mayor que.
- `<=`: Menor o igual que.
- `>=`: Mayor o igual que.

Ejemplo práctico:

```
edad = 18

if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")
```

3. Operadores Lógicos

Los operadores lógicos permiten combinar múltiples condiciones:

- `and`: Todas las condiciones deben ser verdaderas.
- `or`: Al menos una condición debe ser verdadera.
- `not`: Invierte el valor de verdad de una condición.

Ejemplo práctico:

```
# Combinación de condiciones
edad = 25
tiene_permiso = True

if edad >= 18 and tiene_permiso:
    print("Puedes entrar al evento.")
else:
```



```
print("No puedes entrar al evento.")
```

4. Uso de Condicionales Anidados

Las condicionales pueden anidarse para evaluar situaciones más complejas, aunque se recomienda no abusar de ellas para mantener el código legible.

Ejemplo práctico:

```
# Evaluar diferentes rangos de temperatura
temperatura = int(input("¿Cuál es la temperatura actual? "))

if temperatura > 30:
    print("Hace mucho calor.")
    if temperatura > 40:
        print("Es peligroso salir.")
elif temperatura >= 20:
    print("El clima es agradable.")
else:
    print("Hace frío.")
```

5. Ejemplo Completo

A continuación, un ejemplo que combina varios conceptos de condicionales:

```
# Sistema de evaluación de entrada al cine
tiene_ticket = True
edad = int(input("¿Cuántos años tienes? "))
expulsado_del_cine = False

if tiene_ticket and edad >= 18:
```

```
print("Puedes entrar al cine.")
elif not tiene_ticket and edad >= 60:
    print("Puedes entrar gratis por ser mayor de 60.")
elif not expulsado_del_cine:
    print("Puedes entrar porque no has sido expulsado.")
else:
    print("No puedes entrar.")
```

6. Condicionales Ternarias

Python permite escribir condicionales simples en una sola línea, utilizando una expresión llamada condicional ternaria.

Ejemplo práctico:

```
edad = 18
mensaje = "Mayor de edad" if edad >= 18 else "Menor de edad"
print(mensaje)
```

7. Comparación de Múltiples Valores

Puedes usar la palabra clave `in` para verificar si un valor pertenece a una lista, tupla, o cualquier otra secuencia.

Ejemplo práctico:

```
# Verificar si un valor está en una lista
dia = "lunes"

if dia in ["lunes", "martes", "miércoles"]:
    print("Es un día laboral.")
```

```
else:
    print("Es un día de descanso.")
```

Importancia de las Condicionales

Las condicionales son esenciales para que los programas respondan a diferentes escenarios. Sin ellas, los programas serían lineales y rígidos, incapaces de manejar decisiones dinámicas. Aprender a trabajar con condicionales permite diseñar software más flexible e interactivo, desde simples menús hasta complejos sistemas de toma de decisiones.

5 - Comparaciones y Operadores Lógicos en Python

Las comparaciones y operadores lógicos son herramientas esenciales en programación que permiten evaluar condiciones y tomar decisiones. Estas operaciones son fundamentales para estructuras como condicionales y bucles, ya que determinan qué bloques de código se ejecutan.

1. Comparaciones

Los operadores de comparación evalúan dos valores y devuelven un resultado booleano: `True` (verdadero) o `False` (falso).

Operadores de comparación en Python:

Operador	Descripción	Ejemplo	Resultado
<code>==</code>	Es igual a	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	No es igual a	<code>5 != 3</code>	<code>True</code>
<code><</code>	Menor que	<code>3 < 5</code>	<code>True</code>
<code>></code>	Mayor que	<code>5 > 3</code>	<code>True</code>
<code><=</code>	Menor o igual que	<code>3 <= 3</code>	<code>True</code>
<code>>=</code>	Mayor o igual que	<code>5 >= 5</code>	<code>True</code>

Ejemplo práctico:

```
edad = 18

if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")
```

2. Operadores Lógicos

Los operadores lógicos permiten combinar múltiples condiciones en una sola expresión lógica. Son esenciales para evaluar situaciones más complejas.

Operadores lógicos en Python:

Operador	Descripción	Ejemplo	Resultado
<code>and</code>	Todas las condiciones deben ser <code>True</code> .	<code>True and False</code>	<code>False</code>
<code>or</code>	Al menos una condición debe ser <code>True</code> .	<code>True or False</code>	<code>True</code>
<code>not</code>	Invierte el valor de verdad de una condición.	<code>not True</code>	<code>False</code>

Ejemplo práctico:

```
# Combinación con operadores lógicos
edad = 25
tiene_permiso = True

if edad >= 18 and tiene_permiso:
    print("Puedes entrar al evento.")
```

```
else:
    print("No puedes entrar al evento.")
```

3. Uso de Comparaciones y Operadores Lógicos en Condicionales

Los operadores lógicos son especialmente útiles cuando necesitas evaluar varias condiciones al mismo tiempo.

Ejemplo práctico:

```
# Acceso a una sala de cine
tiene_ticket = True
edad = 20
expulsado = False

if tiene_ticket and edad >= 18 and not expulsado:
    print("Puedes entrar al cine.")
else:
    print("No puedes entrar.")
```

4. Comparaciones de Múltiples Valores

Python permite realizar comparaciones compactas y legibles:

- Comparaciones encadenadas:

```
x = 15
if 10 < x < 20:
    print("El número está entre 10 y 20.")
```

- Verificar pertenencia con `in` y `not in`:

```
# Verificar si un valor está en una lista
dia = "lunes"

if dia in ["lunes", "martes", "miércoles"]:
    print("Es un día laboral.")
else:
    print("Es un día de descanso.")
```

5. Ejemplo Completo

Un programa que evalúa múltiples condiciones con operadores lógicos:

```
# Evaluar la elegibilidad para un descuento
edad = int(input("¿Cuál es tu edad? "))
es_estudiante = input("¿Eres estudiante? (sí/no) ").lower() =
= "sí"

if edad < 18 or edad > 65 or es_estudiante:
    print("Eres elegible para un descuento.")
else:
    print("No calificas para el descuento.")
```

6. Importancia de los Operadores Lógicos y Comparaciones

Estos operadores son esenciales para implementar lógica condicional en los programas. Permiten manejar casos complejos, verificar reglas, y diseñar sistemas que se adaptan a múltiples escenarios. Comprenderlos te permitirá desarrollar código más robusto y eficiente.

6- Bucles

Bucles en Python

Los bucles son estructuras de control que permiten ejecutar repetidamente un bloque de código mientras se cumpla una condición o durante un número específico de iteraciones. Son fundamentales para tareas repetitivas, como procesar elementos de una lista, realizar cálculos o generar patrones.

1. ¿Qué son los bucles?

Un bucle es una herramienta que permite repetir un conjunto de instrucciones sin necesidad de escribir el mismo código varias veces. Python ofrece dos tipos principales de bucles:

- `while`: Se ejecuta mientras una condición sea verdadera.
- `for`: Itera sobre una secuencia (listas, cadenas, rangos, etc.).

2. Bucle `while`

El bucle `while` evalúa una condición al inicio de cada iteración. Si la condición es verdadera, el bloque de código se ejecuta. Cuando la condición es falsa, el bucle termina.

Estructura básica:

```
while condición:
    # Código que se ejecuta mientras la condición sea verdadera
```

Ejemplo práctico:

```
# Contar del 1 al 5
contador = 1
while contador <= 5:
```

```
print(contador)
contador += 1 # Incrementar el contador
```

Consideraciones importantes:

- Asegúrate de que la condición cambie durante la ejecución del bucle, para evitar bucles infinitos.
- Puedes usar `break` para salir del bucle y `continue` para saltar a la siguiente iteración.

Ejemplo con `break` y `continue`:

```
contador = 1
while contador <= 10:
    if contador == 5:
        break # Salir del bucle si contador es 5
    print(contador)
    contador += 1
```

3. Bucle `for`

El bucle `for` itera sobre los elementos de una secuencia (como listas, cadenas o rangos). Es útil cuando se conoce el número de iteraciones o se desea recorrer elementos de una colección.

Estructura básica:

```
for elemento in secuencia:
    # Código que se ejecuta por cada elemento
```

Ejemplo práctico:

```
# Iterar sobre una lista
nombres = ["Ana", "Luis", "Pedro"]
for nombre in nombres:
```



```
print(f"Hola, {nombre}")

# Iterar sobre un rango de números
for i in range(1, 6): # Del 1 al 5
    print(i)
```

Métodos comunes con `range()` :

- `range(inicio, fin, paso)` : Genera una secuencia desde `inicio` hasta `fin-1` con un paso opcional.

```
# Contar de 2 en 2
for i in range(0, 10, 2):
    print(i)
```

4. Bucles Anidados

Puedes anidar bucles dentro de otros para trabajar con estructuras complejas, como matrices o patrones.

Ejemplo práctico:

```
# Crear un patrón de números
for i in range(1, 4): # Rango exterior
    for j in range(1, 4): # Rango interior
        print(f"i={i}, j={j}")
```

5. Usando `else` con Bucles

Python permite asociar un bloque `else` a un bucle. Este bloque se ejecuta solo si el bucle no se interrumpe con un `break`.

Ejemplo práctico:

```
for i in range(5):
    if i == 3:
        break
else:
    print("El bucle terminó sin interrupciones.") # No se ejecuta porque hubo un `break`
```

6. Ejemplo Completo

A continuación, un programa que combina bucles `while`, `for` y estructuras de control adicionales:

```
# Juego de adivinanza con un número
import random

numero_secreto = random.randint(1, 10)
intentos = 0

while intentos < 3:
    adivinanza = int(input("Adivina el número (entre 1 y 10):"))
    intentos += 1

    if adivinanza == numero_secreto:
        print("¡Felicidades! Adivinaste el número.")
        break
    elif adivinanza < numero_secreto:
        print("El número es mayor.")
    else:
        print("El número es menor.")
else:
    print(f"Lo siento, el número era {numero_secreto}.")
```

7. Diferencias entre `while` y `for`

- Usa `while` cuando no sabes cuántas iteraciones necesitas y depende de una condición.
 - Usa `for` cuando trabajas con una secuencia o conoces el número exacto de iteraciones.
-

Importancia de los Bucles

Los bucles son herramientas esenciales para automatizar tareas repetitivas y trabajar con colecciones de datos. Comprender cómo usarlos correctamente te permitirá escribir programas más eficientes y manejar problemas más complejos, como procesamiento de datos, simulaciones y generación de reportes.

Complemento y Recursos Adicionales

Además de lo aprendido, es importante que sigan practicando. Python es un lenguaje que se domina con la práctica constante. Les recomiendo visitar [W3Schools Python Tutorial](#) para continuar aprendiendo y profundizando en los temas que hemos visto en clase.

También pueden consultar la documentación oficial de Python en python.org, donde encontrarán una amplia referencia sobre funciones, módulos y ejemplos práctica.

Puedes descargar el editor de código Visual Studio Code desde este [link](#).

Invitación a Seguir Aprendiendo

La programación es una habilidad que se desarrolla con dedicación y curiosidad. Los invito a seguir explorando nuevos lenguajes de programación, frameworks y tecnologías. No se detengan aquí, el mundo de la informática es vasto y las oportunidades son infinitas. ¡Con cada línea de código, están más cerca de convertirse en expertos!

¡Sigán adelante, y recuerden que la clave está en la práctica constante.

Código QR generado con Python

Escuela de Informática y Telecomunicaciones DUOC UC



Repositorio del Curso en GitHub

